

Plano de Testes com GenAI - Robot Framework — API ServeRest

Autor	Gustavo Santos Queiroz
Squad	01
Trilha	AWS & AI for Software Quality Engineering - Nov/2025 - Fase 2
Modelo de AI usada	Gemini 3.1 Pro
Link do repositório	pb.compass_QA-AWS-AI/Documentos/Fase 2/Sprint 4/Challenge 04 - GenAI e Engenharia de Prompt at main · gsqxz/pb.compass_QA-AWS-AI
Prompt Log	prompt_log.md

ANTES VS. DEPOIS

A evolução do plano de testes, impulsionada pelo uso de GenAI, resultou em um salto qualitativo e quantitativo na cobertura da API ServeRest. O cenário inicial contava com 9 casos de teste básicos, focados majoritariamente em caminhos felizes e bugs de regressão. Após o refino, o plano foi expandido para 22 casos , incorporando cenários negativos robustos, validações rigorosas de contrato JSON (schemas) e testes complexos de regras de negócio, como permissões de acesso e integridade de estoque. Esta abordagem elevou a maturidade da suíte, tornando-a capaz de identificar falhas estruturais e de segurança que não seriam detectadas em testes puramente funcionais.

CENÁRIO INICIAL

ID	Título do Cenário	Tipo	Endpoint	Dados de Entrada	Resultado Esperado	Status Code
CT01	Cadastrar Usuário Administrador	Funcional (Positivo)	POST /usuarios	Nome, Email dinâmico, Senha,	Usuário criado com sucesso. Retorna	201 Created

				admini strado r: "true ".	mensag em e _id gerado.	
CT02	Realizar Login (Obter Token)	Funcion al (Positivo)	POST /logi n	Email e Senha válidos utilizado s no CT01.	Autentic ação validada com sucesso. Retorna o Token Bearer (salvo em variável) .	200 OK
CT03	Cadastrar Produto no Estoque	Funcion al (Positivo)	POST /produ tos	Header: Token (CT02). Body: Nome único, Preço válido, Descriçã o, Qtd.	Produto registrad o no sistema. Retorna mensag em de sucesso e o _id do produto.	201 Create d
CT04	Fluxo de Carrinho (Adicionar e Concluir)	Funcion al (Positivo)	POST /carri nhos e DELET E	Header: Token (CT02). Body: idProd uto (CT03) e	Item adiciona do e compra concluíd a. Valida a limpeza	201 e 200

				Quantidade (1).	do carrinho com mensagem de exclusão.	
CT05	Tentativa de Login com Senha Inválida	Funcional (Negativo)	POST /login	Email válido (CT01), porém com senha incorreta (ex: "senha_errada").	Acesso bloqueado. A API retorna a mensagem exata: "Email e/ou senha inválidos".	401 Unauthorized
CT06	Cadastrar Produto com Preço Negativo	Funcional (Negativo)	POST /produtos	Header: Token Admin. Body com campo de preço inválido (preço: -50).	Cadastro rejeitado por violação de regra de negócio. Retorna mensagem exigindo preço positivo.	400 Bad Request
CT07	Validar Contrato de	Contrato	GET /usuari	Nenhum (parâme	Retorna JSON	200 OK

	Lista de Usuários		ios	tros de busca vazios).	válido com as chaves estruturais obrigatórias: quantidade, nome, email, password, administrador e _id.	
CT08	Validar Cadastro de Produto com Preço Decimal	Regressão (Bug-01)	POST /produtos	Header: Token Admin. Body com preco: 50.99.	(Falha Esperada) O sistema deveria aceitar decimais, mas rejeita com erro.	201 (Retorna 400)
CT09	Validar Rejeição com E-mail Inválido	Regressão (Bug-02)	POST /usuarios	Body com e-mail inválido contendo	(Falha Esperada) O sistema	400 (Retorna 201)

				o caracter es especiai s/emojis (#email linval ido 🚀 @qa.co m).	deveria rejeitar o formato, mas aceita indevida mente, criando o usuário.	
--	--	--	--	--	--	--

CENÁRIO REFINADO

CT01 - Cadastrar Usuário Administrador

- **Pré-condições:** Nenhuma. O ambiente deve estar acessível.
- **Massa de Dados:** nome : "QA Tester", email : (gerar um e-mail dinâmico único a cada execução), password : "senha123", administrador : "true".
- **Passo a passo:** Enviar uma requisição POST para o endpoint /usuarios contendo a massa de dados no *body* em formato JSON.
- **Resultado Esperado:** Retornar Status Code **201 Created**. O *body* da resposta deve conter a mensagem "Cadastro realizado com sucesso" e retornar o _id gerado.

CT02 - Realizar Login e Obter Token

- **Pré-condições:** Um usuário válido deve estar cadastrado no banco (executar o CT01 previamente).
- **Massa de Dados:** email e password do usuário cadastrado no CT01.
- **Passo a passo:** Enviar uma requisição POST para o endpoint /login com as credenciais.
- **Resultado Esperado:** Retornar Status Code **200 OK**. O *body* deve conter a mensagem "Login realizado com sucesso" e a chave authorization contendo o token no padrão Bearer. Guardar este token.

CT03 - Cadastrar Produto no Estoque

- **Pré-condições:** Possuir um token de autenticação de um usuário administrador (CT02).
- **Massa de Dados:** nome : (gerar nome único), preco : 150, descricao : "Monitor Dell", quantidade : 10.

- **Passo a passo:** Adicionar o token no *header* `Authorization` . Enviar POST para `/produtos` com a massa de dados.
- **Resultado Esperado:** Retornar Status Code **201 Created**. O *body* deve conter `"Cadastro realizado com sucesso"` e o `_id` do produto.

CT04 - Fluxo de Carrinho (Adicionar e Concluir Compra)

- **Pré-condições:** Usuário logado com token válido e um produto cadastrado com estoque disponível.
- **Massa de Dados:** `idProduto` (obtido no CT03) e `quantidade : 1`.
- **Passo a passo:** 1. Enviar POST para `/carrinhos` com os dados do produto. 2. Enviar DELETE para `/carrinhos/concluir-compra` (requer token no *header*).
- **Resultado Esperado:** O POST deve retornar **201 Created** com `"Cadastro realizado com sucesso"` . O DELETE deve retornar **200 OK** com `"Registro excluído com sucesso"`.

CT05 - Tentativa de Login com Senha Inválida

- **Pré-condições:** Usuário previamente cadastrado.
- **Massa de Dados:** `email` válido, `password : "senha_incorreta_123"`.
- **Passo a passo:** Enviar POST para `/login` com a massa de dados.
- **Resultado Esperado:** Retornar Status Code **401 Unauthorized**. A resposta deve ser exatamente `"Email e/ou senha inválidos"`.

CT06 - Cadastrar Produto com Preço Negativo

- **Pré-condições:** Token de administrador válido.
- **Massa de Dados:** `nome : "Mouse"`, `preco : -50`, `descricao : "Teste QA"`, `quantidade : 10`.
- **Passo a passo:** Enviar POST para `/produtos` .
- **Resultado Esperado:** Retornar Status Code **400 Bad Request**. A API deve bloquear a inserção e retornar a mensagem de validação correspondente (ex: `"preco deve ser um número positivo"`).

CT07 - Validar Contrato de Lista de Usuários

- **Pré-condições:** Nenhuma.
- **Massa de Dados:** Nenhuma (parâmetros de query vazios).
- **Passo a passo:** Enviar requisição GET para `/usuarios` .
- **Resultado Esperado:** Retornar Status Code **200 OK**. A estrutura do JSON de resposta deve obrigatoriamente conter as chaves `quantidade` e o array `usuarios` (com os nós `nome` , `email` , `password` , `administrador` e `_id` como *strings*).

CT08 - [Regressão/Bug] Cadastro de Produto com Preço Decimal

- **Pré-condições:** Token de administrador válido.
- **Massa de Dados:** `preco` : 50.99.
- **Passo a passo:** Enviar POST para `/produtos` .
- **Resultado Esperado [Comportamento Atual]:** Retorna **400 Bad Request** rejeitando a tipagem, mas o esperado para o negócio seria aceitar valores decimais (201 Created).

CT09 - [Regressão/Bug] Rejeição com E-mail em Formato Inválido

- **Pré-condições:** Nenhuma.
- **Massa de Dados:** `email` : "#emailinvalido@qa.com".
- **Passo a passo:** Enviar POST para `/usuarios` .
- **Resultado Esperado [Comportamento Atual]:** A API aceita indevidamente e retorna **201 Created**, quando o esperado seria falhar a validação e retornar **400 Bad Request** bloqueando formatos sem domínio válido.

CT10 - Tentativa de Cadastro de Usuário com E-mail Duplicado

- **Endpoint:** POST `/usuarios`
- **Cenário:** Enviar os mesmos dados de um usuário já existente no banco.
- **Resultado Esperado:** Retornar **400 Bad Request** com a mensagem "Este email já está sendo usado".
- **Visão do QA:** *Testar duplicidade é vital para evitar inconsistência no banco de dados. Um e-mail duplicado pode causar falhas severas de autenticação, onde o sistema não sabe qual conta autorizar durante o login.*

CT11 - Cadastro de Produto por Usuário Sem Permissão (Não-Admin)

- **Endpoint:** POST `/produtos`
- **Cenário:** Logar com um usuário onde `administrador` é "false" e tentar criar um produto.
- **Resultado Esperado:** Retornar **403 Forbidden** com a mensagem "Rota exclusiva para administradores".
- **Visão do QA:** *Isso valida as regras de Autorização (não confundir com Autenticação). Precisamos garantir que usuários comuns não consigam escalar privilégios e alterar o estoque do sistema de forma maliciosa.*

CT12 - Cadastro de Produto com Nome Duplicado

- **Endpoint:** POST `/produtos`
- **Cenário:** Tentar criar um novo produto usando o nome exato de um produto já cadastrado.

- **Resultado Esperado:** Retornar **400 Bad Request** com a mensagem "Já existe produto com esse nome".
- **Visão do QA:** Em um e-commerce, IDs diferentes para o mesmo nome de produto quebram as buscas, criam duplicidade no catálogo e afetam integrações financeiras. O bloqueio tem que ocorrer na raiz.

CT13 - Login com Campo de E-mail Ausente (Payload Incompleto)

- **Endpoint:** POST /login
- **Cenário:** Enviar o JSON apenas com a password, omitindo totalmente a chave email.
- **Resultado Esperado:** Retornar **400 Bad Request** com a mensagem "email é obrigatório".
- **Visão do QA:** Isso testa a resiliência da API contra requisições malformadas. O backend nunca deve tentar processar (ou estourar um erro 500 Interno) um login se faltam dados vitais no contrato da requisição.

CT14 - Adicionar Produto ao Carrinho com Quantidade Maior que o Estoque

- **Endpoint:** POST /carrinhos
- **Cenário:** Pegar um produto que tem quantidade: 2 no estoque e tentar adicionar quantidade: 5 no carrinho.
- **Resultado Esperado:** Retornar **400 Bad Request** com a mensagem "Produto insuficiente em estoque".
- **Visão do QA:** Esse é um teste de regra de negócio puro. Se a API permitir isso, a empresa estará vendendo produtos fantasmas que não tem para entregar, gerando passivos graves com os clientes.

CT15 - Cadastrar Usuário Cliente (Não-Administrador)

- **Pré-condições:** Nenhuma. O ambiente deve estar acessível.
- **Massa de dados:** nome: "Cliente Fiel", email: (gerar um e-mail dinâmico único), password: "senha123", administrador: "false".
- **Passo a passo:** Enviar uma requisição POST para o endpoint /usuarios com a massa de dados no formato JSON.
- **Resultado Esperado:** A API deve retornar o Status Code **201 Created**. O corpo da resposta deve conter a mensagem "Cadastro realizado com sucesso" e o respectivo _id do novo usuário cliente.

CT16 - Editar um Produto Existente (Atualizar Preço)

- **Pré-condições:** Estar logado com um usuário Administrador (ter o Token) e possuir o _id de um produto já cadastrado.

- **Massa de dados:** Usar o mesmo `nome` do produto original (para não criar um novo), alterando o `preco` de 150 para 180.
- **Passo a passo:** Adicionar o Token no Header `Authorization` . Enviar uma requisição PUT para o endpoint `/produtos/{_id}` com os novos dados no corpo da requisição.
- **Resultado Esperado:** A API deve retornar o Status Code **200 OK**. O corpo da resposta deve conter a mensagem `"Registro alterado com sucesso"` .

CT17 - Cancelar Compra e Restaurar Estoque

- **Pré-condições:** O usuário deve estar logado (ter o Token) e possuir um carrinho ativo com produtos dentro.
- **Massa de dados:** Nenhuma no corpo da requisição (apenas o Token no Header).
- **Passo a passo:** Enviar uma requisição DELETE para o endpoint `/carrinhos/cancelar-compra` .
- **Resultado Esperado:** A API deve retornar o Status Code **200 OK**. O corpo da resposta deve confirmar o estorno contendo a mensagem `"Registro excluído com sucesso. Estoque dos produtos reabastecido"` .

CT18 - Criar Novo Carrinho Após Cancelamento

- **Pré-condições:** O usuário deve ter acabado de cancelar um carrinho com sucesso (execução do CT17).
- **Massa de dados:** O `idProduto` de um produto válido com estoque e a `quantidade` : 2.
- **Passo a passo:** Enviar uma requisição POST para `/carrinhos` contendo os dados do novo produto que deseja comprar.
- **Resultado Esperado:** A API deve retornar o Status Code **201 Created**. O corpo da resposta deve conter a mensagem `"Cadastro realizado com sucesso"` . *(Isso prova que a regra de carrinho único foi resetada e liberou o usuário para comprar novamente).*

CT19 - Excluir um Usuário Sem Vínculos (Sem Carrinho Ativo)

- **Pré-condições:** Ter o `_id` de um usuário recém-criado que não tenha nenhum carrinho cadastrado no sistema no momento.
- **Massa de dados:** O `_id` do usuário.
- **Passo a passo:** Enviar uma requisição DELETE para o endpoint `/usuarios/{_id}` .
- **Resultado Esperado:** A API deve retornar o Status Code **200 OK**. O corpo da resposta deve exibir a mensagem `"Registro excluído com sucesso"` .

CT20 - Validar o Contrato da Lista de Produtos

- **Pré-condições:** A API deve estar acessível (idealmente, com pelo menos um produto cadastrado para validar o interior do array, mas o teste valida a estrutura de qualquer forma).
- **Massa de dados:** Nenhuma (requisição vazia sem parâmetros).
- **Passo a passo:** Enviar uma requisição GET para o endpoint `/produtos`.
- **Resultado Esperado:** A API deve retornar o Status Code **200 OK**. A estrutura (Schema) do JSON de resposta deve obrigatoriamente validar as seguintes regras:
 - A raiz deve conter a chave `quantidade` do tipo **Integer**.
 - A raiz deve conter a chave `produtos` do tipo **Array**.
 - Dentro do array `produtos`, os objetos devem conter as chaves obrigatórias: `nome` (String), `preco` (Integer), `descricao` (String), `quantidade` (Integer) e `_id` (String).

CT21 - Validar o Contrato de Resposta do Login

- **Pré-condições:** Um usuário válido deve estar previamente cadastrado no banco de dados.
- **Massa de dados:** `email` e `password` válidos de um usuário existente.
- **Passo a passo:** Enviar uma requisição POST para o endpoint `/login` com as credenciais válidas no corpo da requisição.
- **Resultado Esperado:** A API deve retornar o Status Code **200 OK**. O Schema da resposta deve conter estritamente:
 - A chave `message` do tipo **String**.
 - A chave `authorization` do tipo **String**.
 - *(Regra extra de contrato):* O valor de `authorization` deve obrigatoriamente iniciar com o prefixo `"Bearer "`.

CT22 - Validar o Contrato do Carrinho de Compras

- **Pré-condições:** Nenhuma (busca geral de carrinhos).
- **Massa de dados:** Nenhuma.
- **Passo a passo:** Enviar uma requisição GET para o endpoint `/carrinhos`.
- **Resultado Esperado:** A API deve retornar o Status Code **200 OK**. O Schema da resposta JSON deve ser validado contra os seguintes tipos:
 - `quantidade` deve ser **Integer**.
 - `carrinhos` deve ser um **Array**.
 - Os objetos dentro de `carrinhos` devem ter os metadados: `precoTotal` (Integer), `quantidadeTotal` (Integer), `idUsuario` (String) e `_id` (String).
 - A chave interna `produtos` (dentro de cada carrinho) deve ser um **Array** de objetos contendo: `idProduto` (String), `quantidade` (Integer) e `precoUnitario` (Integer).

MATRIZ DE COBERTURA DE TESTES

Endpoint	Testes Positivos (Caminho Feliz)	Testes Negativos (Erros e Limites)	Testes de Contrato (Schemas)
/usuarios	CT01, CT15 🤖	CT09, CT10 🤖, CT19	CT07
/login	CT02 🤖	CT05 🤖, CT13	CT21 🤖
/produtos	CT03 🤖, CT16 🤖	CT06, CT08, CT11 🤖, CT12 🤖	CT20 🤖
/carrinhos	CT04, CT18	CT14, CT17	CT22

LISTA DE TESTES AUTOMATIZADOS

📁 Suíte de Testes Positivos (tests/test_positivos.robot)

- [CT15] Cadastrar Usuário Cliente com Sucesso
- [CT02] Realizar Login com Sucesso
- [CT03] Cadastrar Produto com Sucesso
- [CT16] Editar um Produto Existente

📁 Suíte de Testes Negativos (tests/test_negativos.robot)

- [CT05] Tentativa de Login com Senha Inválida
- [CT10] Tentativa de Cadastro de Usuário com E-mail Duplicado
- [CT11] Cadastro de Produto por Usuário Sem Permissão (Não-Admin)
- [CT12] Cadastro de Produto com Nome Duplicado

📁 Suíte de Testes de Contrato (tests/test_contrato.robot)

- [CT20] Validar Contrato da Lista de Produtos
- [CT21] Validar Contrato de Resposta do Login